Software Re-Engineering

Lecture 7

Re-architecting

Instructor:

Dr. Natalia Chaudhry,

Assistant Professor, Software Engineering Department, PU.

SE Fall 2020

Agenda

- Splitting a monolithic codebase into multiple components
- Distributing a web application into a collection of services
- The pros and cons of microservices



Re-architecturing

Re-architecting is refactoring at a higher level than that of methods and classes.

When refactoring, you might move some classes into a separate package, whereas re-architecting might involve moving them out of the main codebase and into a separate library.



Imagine you have a monolithic e-commerce application with the following structure:

```
ecommerce-app/
|
├── src/
|   ├── main/
|   |   ├── java/
|   |   |   ├── com/
|   |   |   |   ├── ecommerce/
|   |   |   |   |   ├── order/
|   |   |   |   |   |   ├── Order.java
|   |   |   |   |   |   ├── OrderService.java
|   |   |   |   |   ├── payment/
|   |   |   |   |   |   ├── Payment.java
|   |   |   |   |   |   ├── PaymentService.java
|   |   |   |   |   ├── user/
|   |   |   |   |   |   ├── User.java
|   |   |   |   |   |   ├── UserService.java
|   |   |   |   |   ├── MainApplication.java
```

Refactoring: Moving Classes into a Separate Package

Refactoring involves improving the internal structure of the code without changing its external behavior. In this case, you might decide to reorganize the code to improve modularity and maintainability.

You notice that the order and payment packages are tightly coupled and want to separate them for better clarity. You decide to create a new package called transaction to group related classes.

BEFORE

```
ecommerce-app/
├── src/
│   ├── main/
│   │   ├── java/
│   │   │   ├── com/
│   │   │   │   ├── ecommerce/
│   │   │   │   │   ├── order/
│   │   │   │   │   │   ├── Order.java
│   │   │   │   │   │   ├── OrderService.java
│   │   │   │   │   ├── payment/
│   │   │   │   │   │   ├── Payment.java
│   │   │   │   │   │   ├── PaymentService.java
```

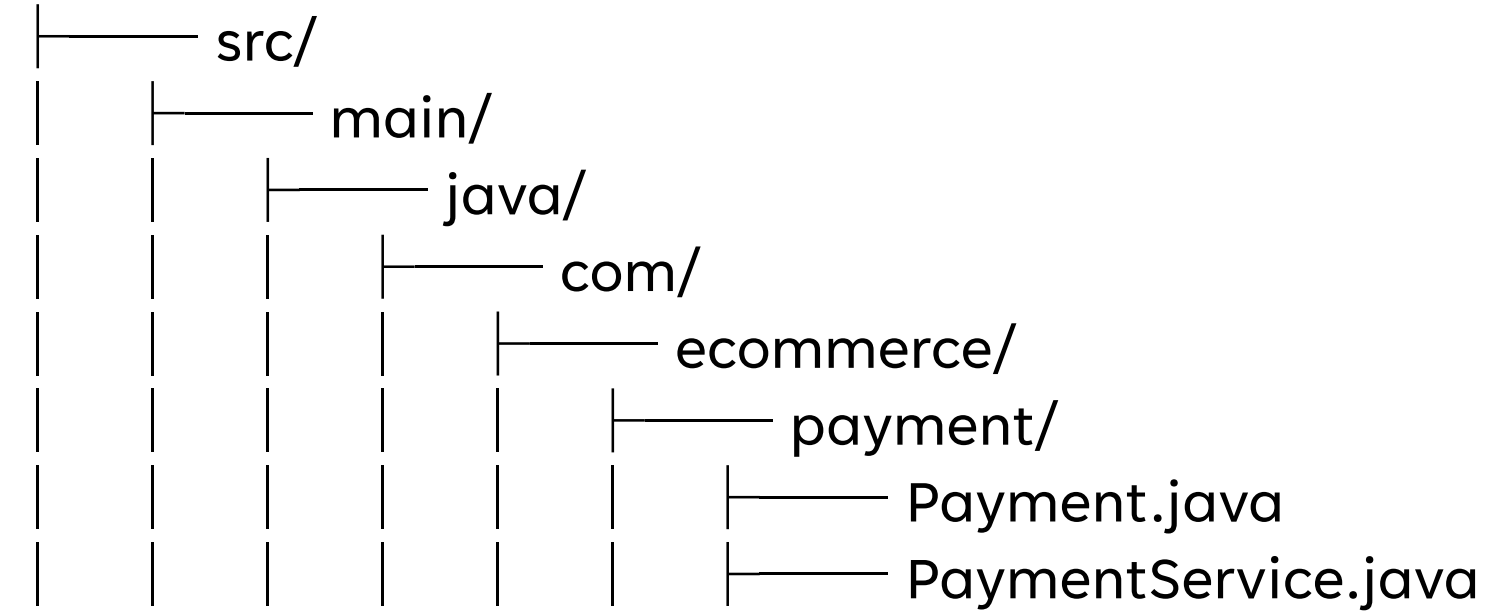
BEFORE

```
ecommerce-app/
├── src/
│   ├── main/
│   │   ├── java/
│   │   │   ├── com/
│   │   │   │   ├── ecommerce/
│   │   │   │   │   ├── transaction/
│   │   │   │   │   │   ├── order/
│   │   │   │   │   │   │   ├── Order.java
│   │   │   │   │   │   │   ├── OrderService.java
│   │   │   │   │   │   ├── payment/
│   │   │   │   │   │   │   ├── Payment.java
│   │   │   │   │   │   │   ├── PaymentService.java
```

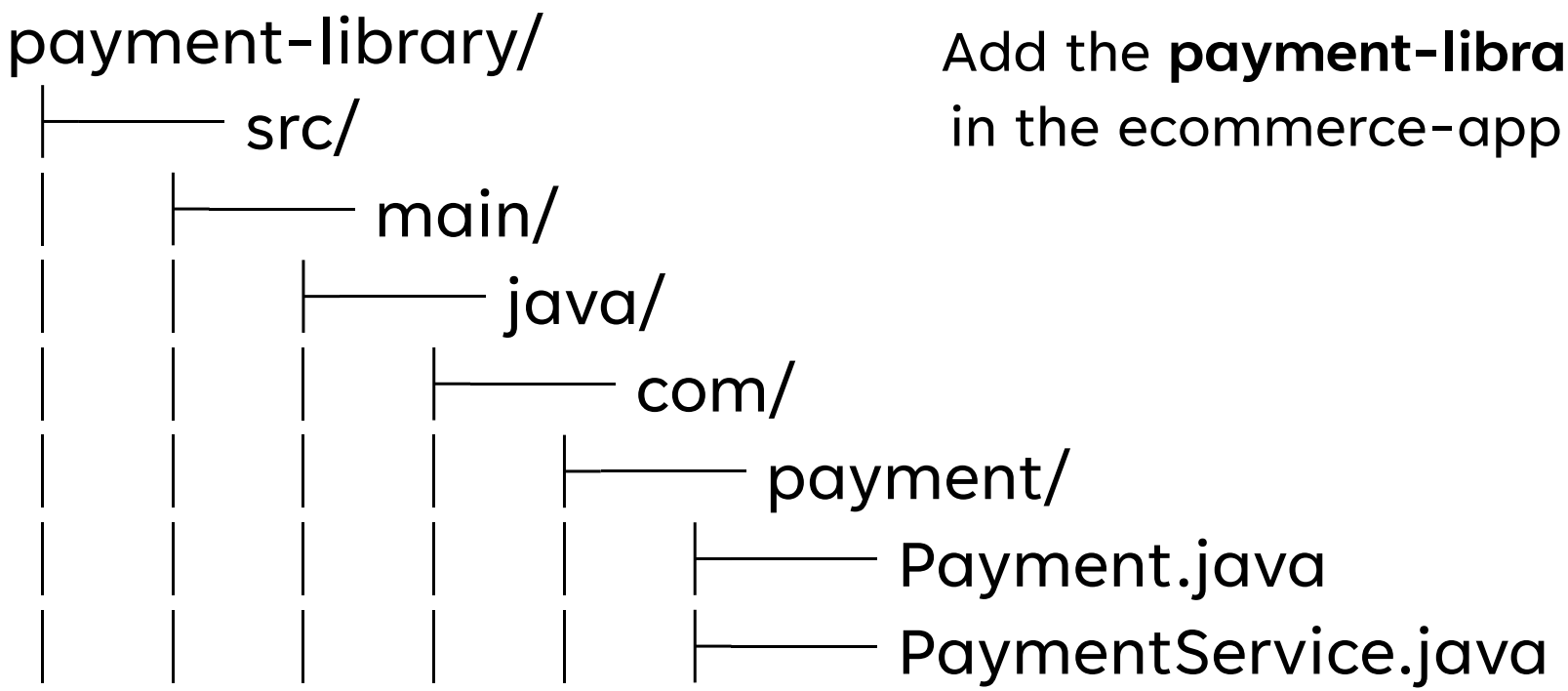
Re-architecting: Moving Classes into a Separate Library
Re-architecting involves making significant structural changes to the system, often to improve scalability, maintainability, or reusability. This might involve extracting parts of the codebase into a separate library or service.

Example:
You realize that the payment functionality is used across multiple projects and decide to extract it into a standalone library so it can be reused independently.

BEFORE


ecommerce-app/
src/
main/
java/
com/
ecommerce/
payment/
Payment.java
PaymentService.java

AFTER



Add the **payment-library** as a dependency in the ecommerce-app build configuration

Benefits..

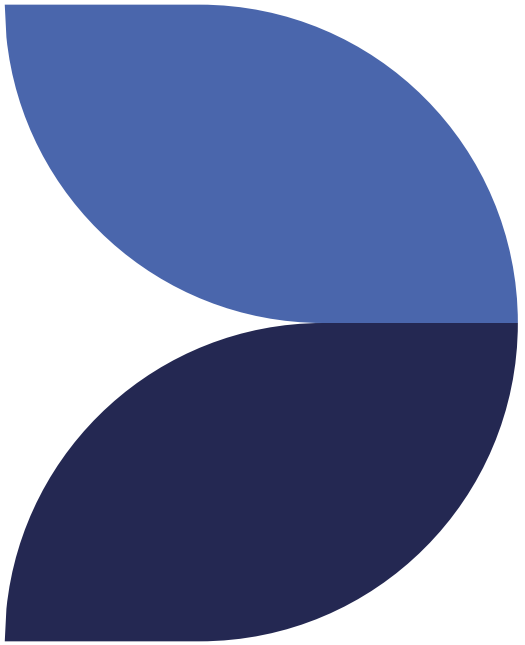
- Quality through modularity (low defects density)
- Maintainability through good design (separation of concerns)
- Autonomy through independence (separate team of developers)
- Promotes code reuse across projects.

Drawbacks..

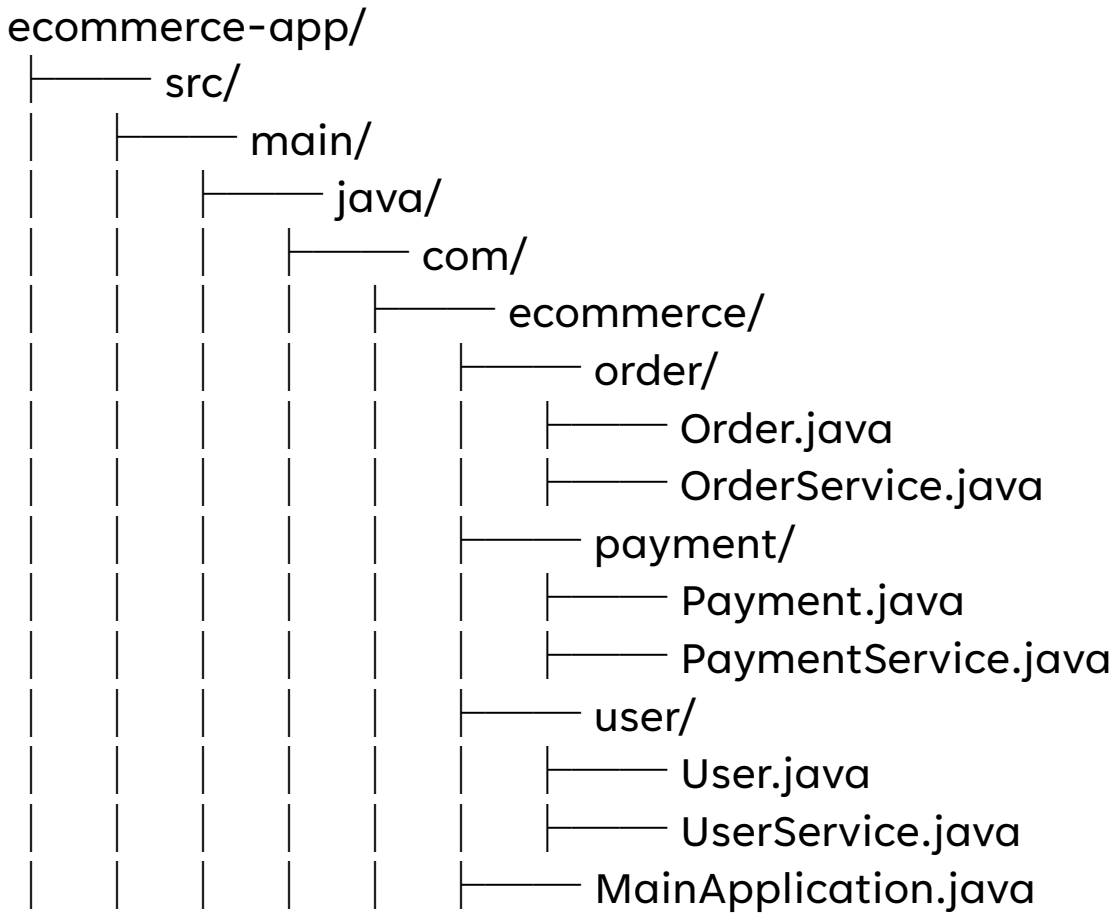
- The management of source code will become more complex, because there are now multiple modules instead of just one.
- You'll need to manage the building of all these modules, perhaps versioning them separately, and package them together, so your build scripts and workflows will become more numerous and/or complex.



Terminologies



- **Monolithic codebase—**A codebase in which all source code is managed in one folder and built as one binary file. In your IDE, everything is in one project. In Java terms, everything is in the same JAR file.



Everything (e.g., business logic, database access, user interface) is part of one large application.

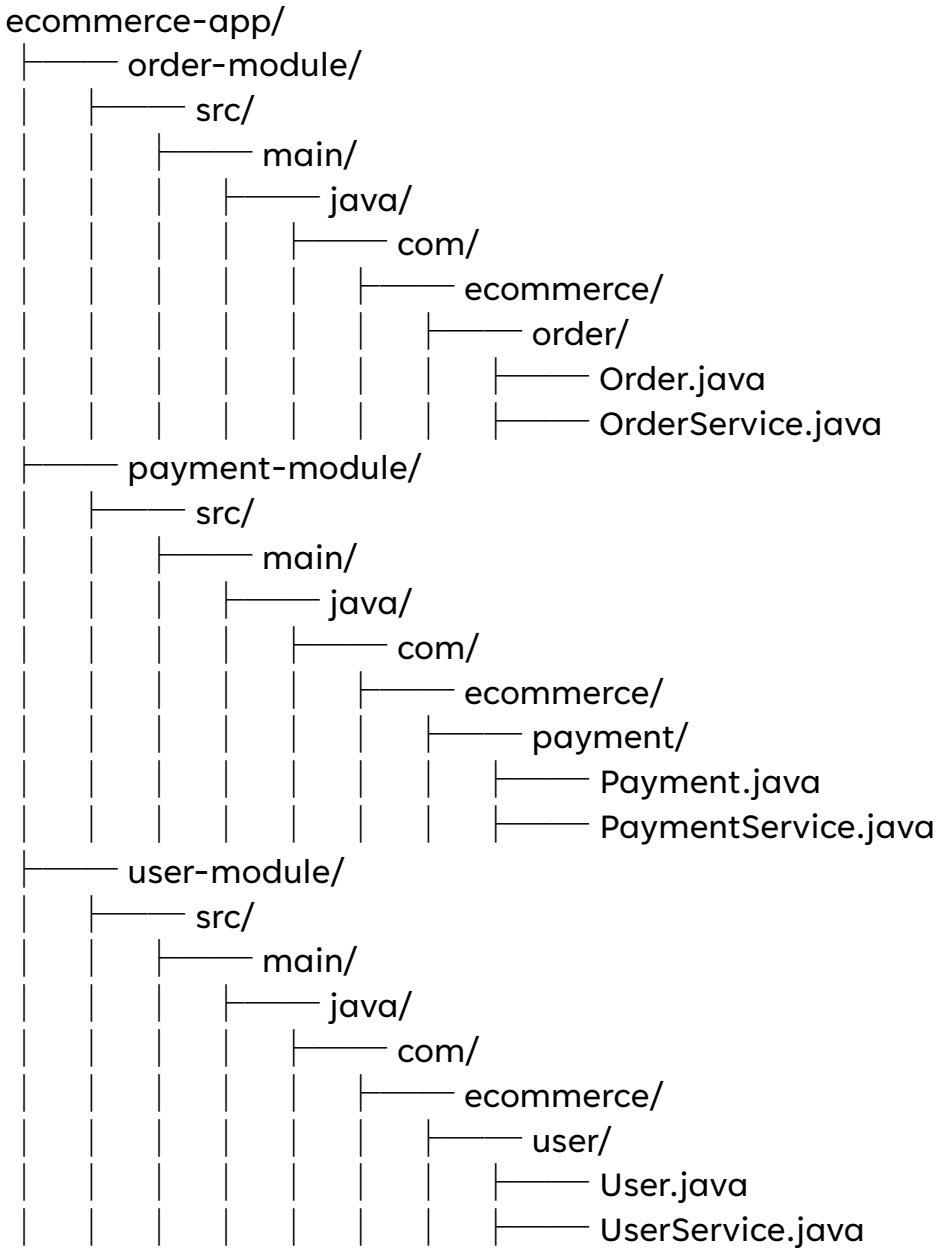
Pros:

- Simpler to develop and test initially.
- Easier to deploy since there’s only one artifact.
- No need to manage inter-module dependencies.

Cons:

- Harder to maintain as the codebase grows.
- Tight coupling makes it difficult to isolate bugs or make changes.
- Scaling is inefficient since the entire application must be scaled.

- **Module (or component)—**A modular codebase is divided into smaller, independent modules or components. Each module has a specific responsibility and can be developed, tested, and deployed independently. Modules can be part of the same codebase (modular monolith) or exist as separate libraries or services (microservices).


ecommerce-app/
order-module/
src/
main/
java/
com/
ecommerce/
order/
Order.java
OrderService.java
payment-module/
src/
main/
java/
com/
ecommerce/
payment/
Payment.java
PaymentService.java
user-module/
src/
main/
java/
com/
ecommerce/
user/
User.java
UserService.java

Pros:

- Easier to maintain and extend.
- Promotes reusability and separation of concerns.
- Teams can work independently on different modules.
- Better scalability since only specific modules can be scaled.

Cons:

- Increased complexity in managing dependencies between modules.
- Requires more upfront design and planning.
- Deployment can be more complex, especially in distributed systems (e.g., microservices).

Aspect	Monolithic Codebase	Modular Codebase
Structure	Single, unified codebase.	Divided into independent modules.
Coupling	Tightly coupled components.	Loosely coupled components.
Deployment	Deployed as a single unit.	Modules can be deployed independently.
Scalability	Scaling requires scaling the entire application.	Individual modules can be scaled.
Development	Easier to start but harder to maintain over time.	Requires more upfront design but easier to maintain.
Reusability	Limited reusability.	Modules can be reused across projects.
Complexity	Simpler initially, but complexity grows over time.	More complex initially but manageable as the system grows.

When to Use Each Approach

Monolithic Codebase:

- Small to medium-sized applications.
- Projects with a small team or limited resources.
- When simplicity and quick development are priorities.
- When the application doesn’t require frequent scaling or updates.

Modular Codebase:

- Large, complex applications.
- Projects with multiple teams working on different features.
- When reusability and scalability are important.
- When the application is expected to grow significantly over time.

Monolithic application—An application that runs entirely in one process on one machine. It may be built from either a monolithic codebase or a collection of modules.

- **API -** API is the acronym for application programming interface — a software intermediary that allows two applications to talk to each other. API architecture is usually explained in terms of client and server. The application sending the request is called the client, and the application sending the response is called the server.

Monolithic application

Monolithic application combines the user interface and data access layers for multiple features into one application.

Monolithic systems can be a good fit for smaller, less complex applications that don’t need to quickly scale or be routinely maintained.

Example for Monolithic Approach

- Consider an example of Ecommerce application, that authorizes customer, takes an order, check products inventory, authorize payment and ships ordered products.
- This application consists of several components including e-Store User interface for customers (Store web view) along with some backend services to check products inventory, authorize and charge payments and shipping orders.
- Despite having different components/modules/services, the application is built and deployed as one Application for all platforms (i.e. desktop, mobile and tablet) using RDBMS as a data source.

- **Microservices** are an approach to application development in which a large application is built as a suite of modular services (i.e. loosely coupled modules/components). Each module supports a specific business goal and uses a simple, well-defined interface to communicate with other sets of services.
- Instead of sharing a single database as in Monolithic application, each microservice has its own database.

Consider the same example of the e-commerce application, which consists of several components/modules.
Define each component/module as a separate loosely coupled service depending on the requirement, which may collaborate with each other based on the scenario.

- Authorization Service — Responsible for authorizing customer.
- Order Service — takes an order and process it.
- Catalog Service — Manage products and check products inventory.
- Cart Service — Manage user cart, this service can utilize Catalog service as a data source.
- Payment Service — Manage and Authorize payments.
- Shipping Service — Ships ordered products.

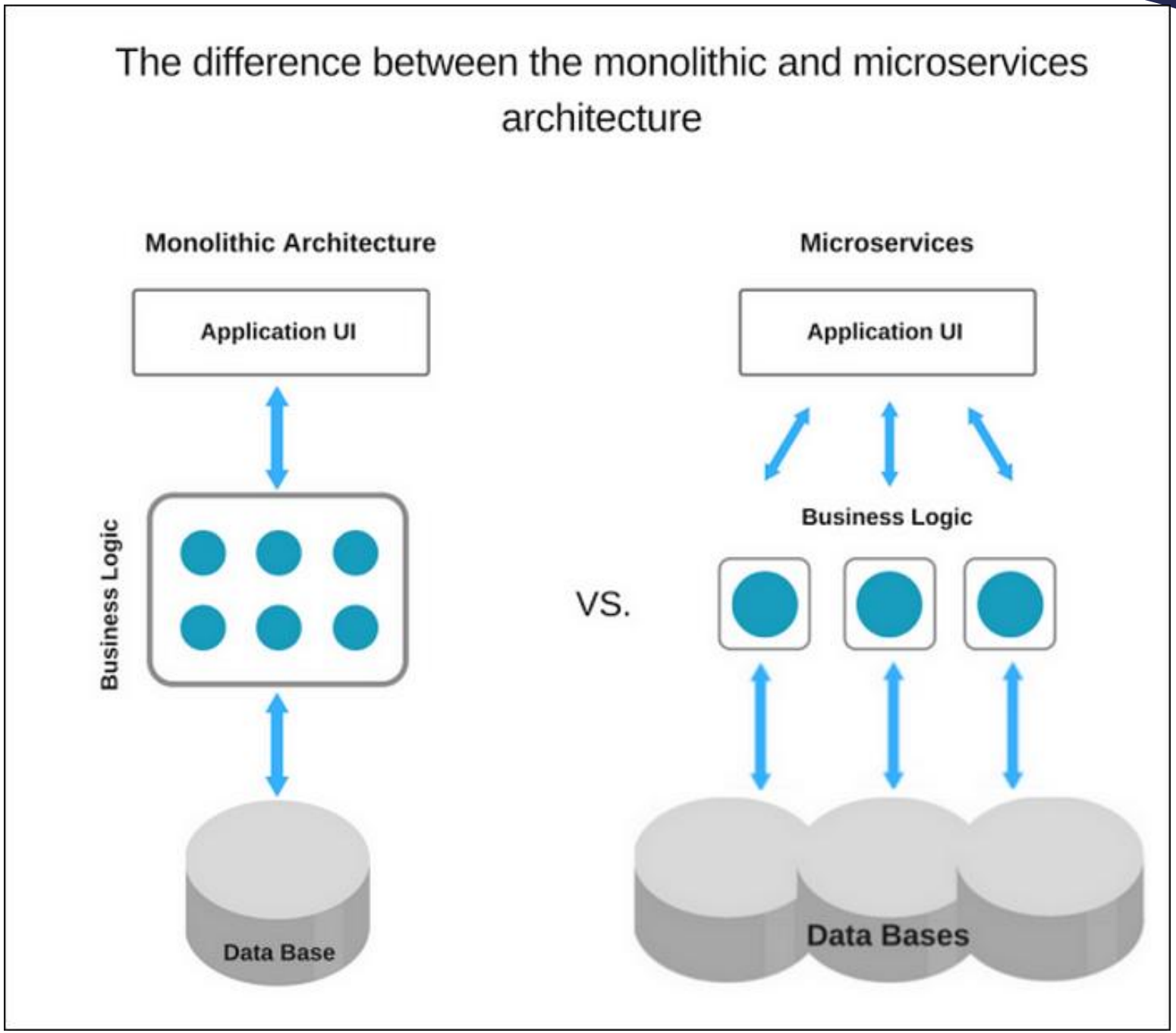
Benefits..

- Microservices Enables the continuous delivery and deployment of large, complex applications.
- Better testability — services are smaller and faster to test.
- Better deployability — services can be deployed independently.
- It enables you to organize the development effort around multiple teams.
- Comfortable for a developer to understand
- The IDE is faster making developers more productive
- The application starts faster, which makes developers more productive, and speeds up deployments
- Improved fault isolation. For example, if there is a memory leak in one service then only that service is affected. The other services continue to handle requests.

- **Services are lightweight**. Since services have a fraction of the capabilities of a complete monolithic application, they're a fraction of the size. This makes it easier to run multiple services on a single host, migrate a service between hosts, and restart a service quickly—often just a few seconds compared to several minutes for a monolith.
- **Services have discrete and clearly defined boundaries**. Each service performs a specific function that no other service provides. It exposes this functionality to other services via API, RPC, or similar so that other services can consume it. Services don't need to be aware of what other services are doing, only how to interact with them.
- **Services are easily scalable**. Services should be easy to scale and replicate. We should be able to increase the total compute capacity available to a service by deploying additional instances of it, and we should be able to add or remove instances without impacting throughput.

Drawbacks..

- Developers must deal with the additional complexity of creating a distributed system.
- Developer tools/IDEs are oriented on building monolithic applications and don’t provide explicit support for developing distributed applications.
- Testing is more difficult as compared to Monolith applications.
- Developers must implement the inter-service communication mechanism.


The difference between the monolithic and microservices architecture
Monolithic Architecture
Application UI
Business Logic
Data Base
VS.
Microservices
Application UI
Business Logic
Data Bases

Monolithic to Microservices

- GraphQL is an open-source data query language and a server-side runtime for Application Programming Interfaces (APIs).
- It has been built to make APIs fast and flexible for developers, allowing them to create requests that pull data from multiple sources in a single API call.
- GraphQL is a query language for APIs that allows clients to request exactly the data they need, reducing over-fetching and under-fetching of data, which are common issues with traditional REST APIs.

A **REST API (Representational State Transfer Application Programming Interface)** is an architectural approach for designing networked applications, particularly web services. It allows different software systems to communicate over the internet using standard HTTP methods.

HTTP Methods in REST API:

Method	Description	Use Case
GET	Retrieve data from the server	Fetching a list of users
POST	Submit data to the server to create a resource	Creating a new user
PUT	Update an existing resource entirely	Updating user details
PATCH	Partially update an existing resource	Updating user email only
DELETE	Remove a resource from the server	Deleting a user



- **Endpoints** in the context of a **REST API** are specific **URLs** through which a client can access the resources or services provided by the server.
- They define where the API can be accessed and which operations can be performed.



What Makes an Endpoint?

- An endpoint typically consists of:
- **Base URL:** The root address of the API.
- **Example:** https://api.example.comResource Path: Specifies the resource being accessed.Example: /users or /products/123HTTP
- **Method:** Defines the type of action to be performed (e.g., GET, POST, PUT, DELETE).



Scenario: Fetching User Data

Imagine you’re building a social media app, and you need to fetch user data, including their name, email, and a list of their posts (with titles and content).



Traditional REST API Approach

In a REST API, you might have two endpoints:

/users/:id - Fetches user details (name, email, etc.).

/users/:id/posts - Fetches the user's posts.

Problem 1: Over-fetching

If you only need the user's name and email, the /users/:id endpoint might still return additional unnecessary data (e.g., address, phone number, etc.).



Problem 2: Under-fetching

To get both the user's details and their posts, you'd need to make two separate requests:

GET /users/1 → Fetches user details.

GET /users/1/posts → Fetches user's posts.

This can lead to multiple round trips to the server, increasing latency and reducing efficiency.

With GraphQL, you can request exactly the data you need in a single query.



GraphQL Query

```
query {
  user(id: 1) {
    name
    email
    posts {
      title
      content
    }
  }
}
```

Response

```
{
  "data": {
    "user": {
      "name": "John Doe",
      "email": "john@example.com",
      "posts": [
        {
          "title": "My First Post",
          "content": "This is the content of my first post."
        },
        {
          "title": "Another Post",
          "content": "This is another post."
        }
      ]
    }
  }
}
```

Advantages of GraphQL

Efficiency:

No over-fetching: You only get the fields you request (e.g., name, email, posts).

No under-fetching: You can fetch related data (e.g., user and their posts) in a single request.

Flexibility:

- Clients can request different combinations of fields based on their needs.
- For example, another client might only need the user's name and the number of posts:

```
query {
  user(id: 1) {
    name
    posts {
      id
    }
  }
}
```

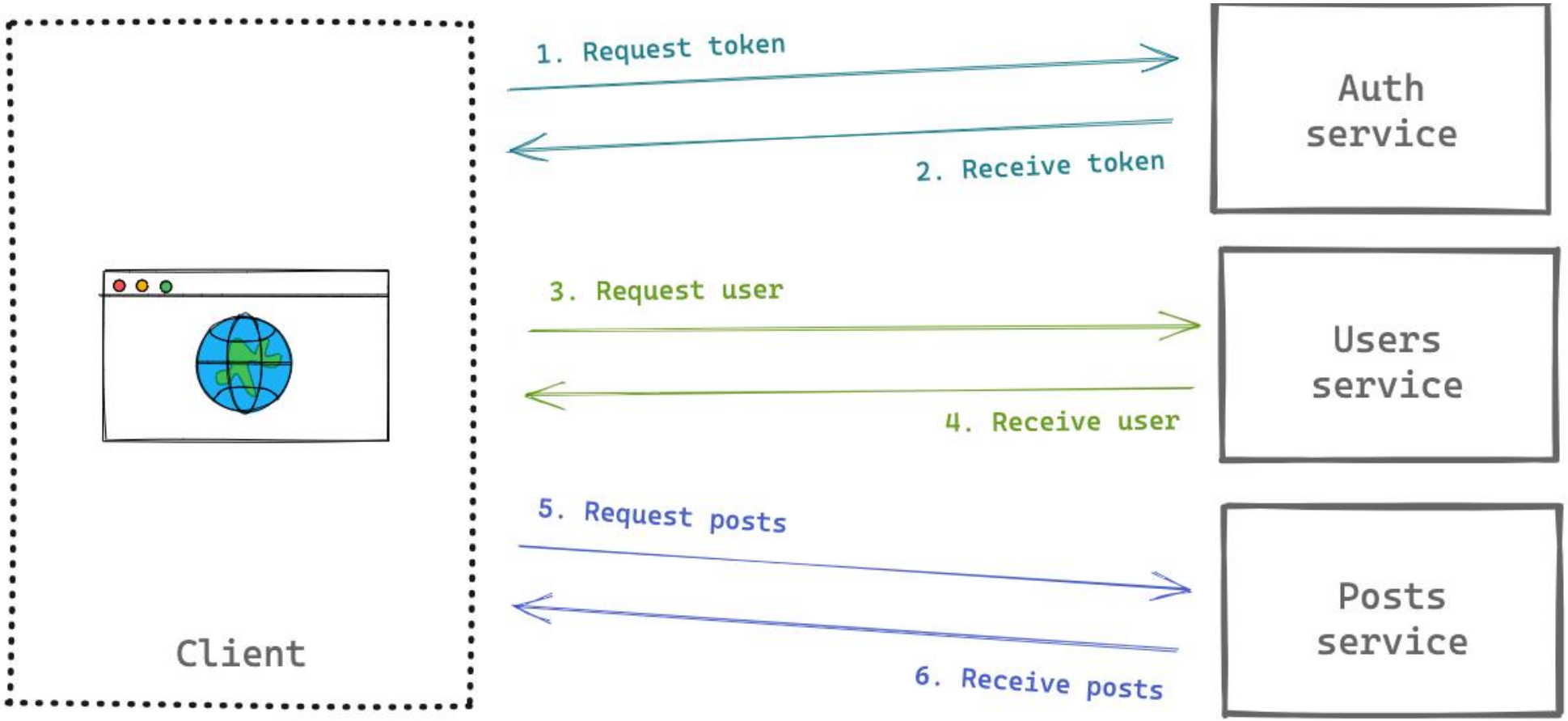
Powerful:

1. GraphQL supports nested queries, allowing you to fetch deeply related data in one request.
2. You can also use features like variables, fragments, and aliases to make queries more dynamic and reusable.



GraphQL as Data Layer

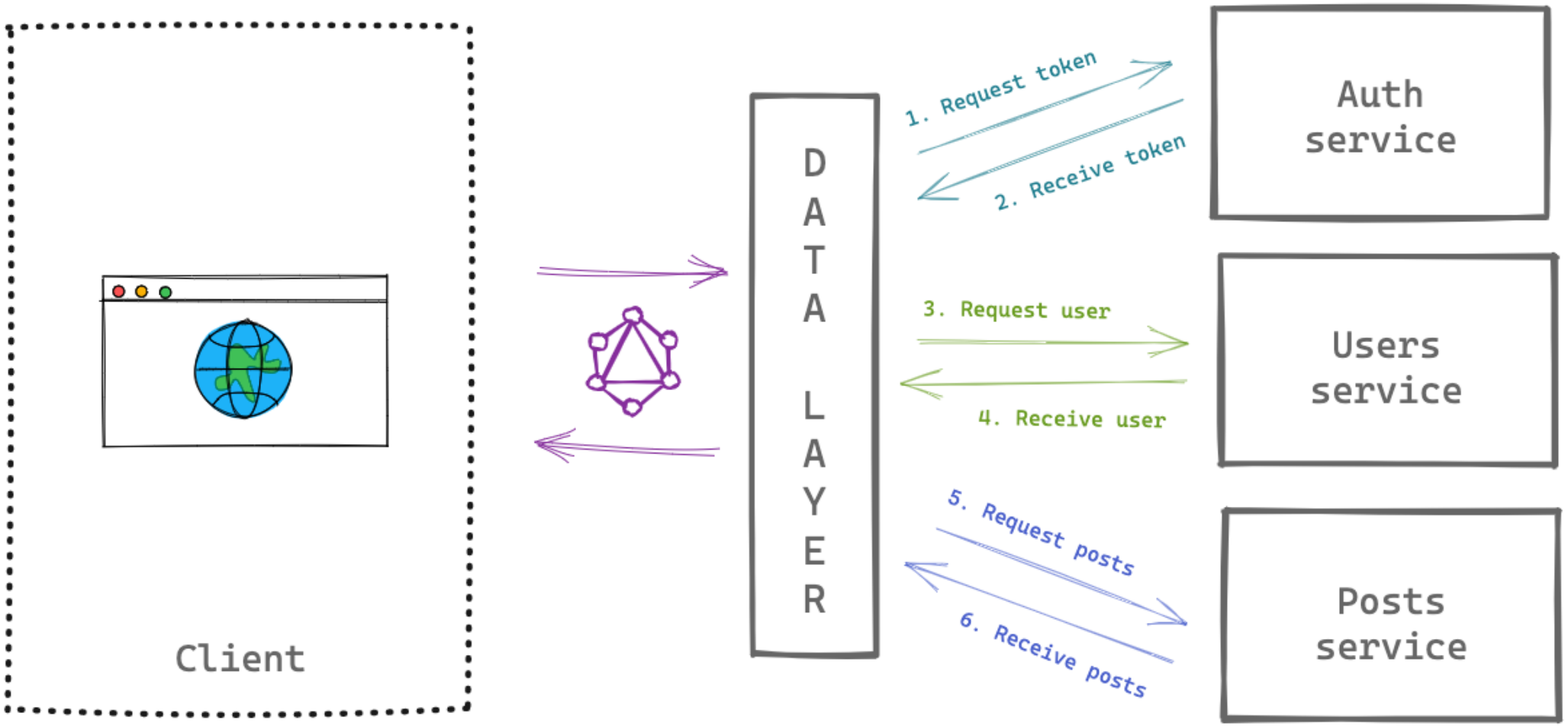
- Suppose you have three microservices in your architecture, services 1 to 3. Or a service for *authentication*, *users* and *posts*.
- In your client, you want to authenticate a user and show all the posts of this user.



The first service a client always needs to interact with is the authentication service.
This service can get an authentication token that is required to verify the requests sent to the other two services.

Without an API data layer that bundles all the APIs of the microservices, you'll send and receive at least six network requests as microservices are not in direct contact with other microservices.

With GraphQL, you can construct a query to retrieve both the user and posts of that user in one request. The API data layer will send the correct requests to the microservices for users and posts and stitch the data into one response.


Client
DATA LAYER
1. Request token
2. Receive token
3. Request user
4. Receive user
5. Request posts
6. Receive posts
Auth service
Users service
Posts service

GraphQL - Application Components

- Server-side Components
- Client-side Components

Server-Side Components

GraphQL server forms the core component on the server side and allows to parse the queries coming from GraphQL client applications.
Apollo Server is most commonly used implementation of GraphQL specification.

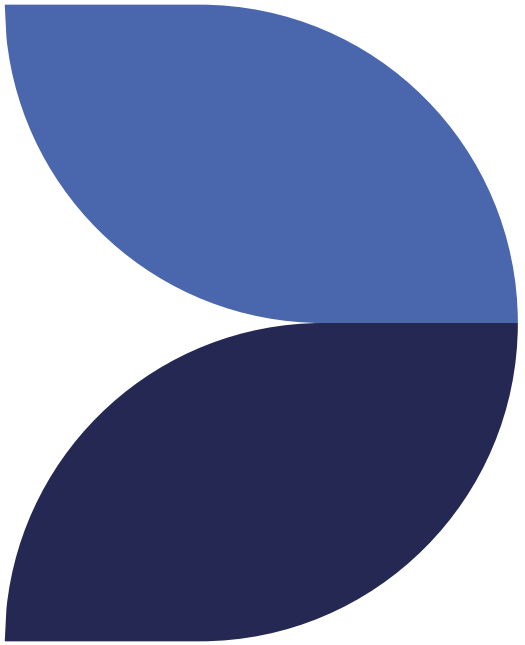
Sr.No.	Server Essentials & Description
1	**Schema** A GraphQL schema is at the center of any GraphQL server implementation and describes the functionality available to the clients which connect to it.
2	**Query** A GraphQL query is the client application request to retrieve data from database or legacy API's.
3	**Resolver** Resolvers provide the instructions for turning a GraphQL operation into data. They resolve the query to data by defining resolver functions.

Client-side Components

Given below are the client-side components

Sr.No.	Tool & Description
1	**GraphiQL** Browser based interface for editing and testing GraphQL queries and mutations.
2	**ApolloClient** Best tool to build GraphQL client applications. Integrates well with all javascript front-end.

types and fields



types and fields

A GraphQL service is created by defining types and fields on those types

Type System defines various data types that can be used in a GraphQL application. The type system helps to define the schema, which is a contract between client and server. The commonly used GraphQL data types are as follows –

Sr.No.	Types & Description
1	**Scalar** Stores a single value
2	**Object** Shows what kind of object can be fetched
3	**Query** Entry point type to other specific types
4	**Mutation** Entry point for data manipulation
5	**Enum** Useful in a situation where you need the user to pick from a prescribed list of options

Scalar Type

Scalar types are primitive data types that can store only a single value. The default scalar types that GraphQL offers are –

Int – Signed 32-bit Integer
Float – Signed double precision floating point value
String – character sequence
Boolean – True or false
ID – A unique identifier, often used as a unique identifier to fetch an object or as the key for a cache.

The syntax for defining a scalar type is as follows –
field: data_type

field named greeting which returns String value.
greeting: String

Object Type

The object type is the most common type used in a schema and represents a group of fields. Each field inside an object type maps to another type, thereby allowing nested types. In other words, an object type is composed of multiple scalar types or object types.

The syntax for defining an object type is given below –

```
type object_type_name
{
   field1: data_type
   field2:data_type
   ....
   fieldn:data_type
}
```

```
type Student {
   stud_id:ID
   firstname: String
   age: Int
   score:Float
}
```

Query Type

A GraphQL query is used to fetch data. It is like requesting a resource in REST-based APIs. To keep it simple, the Query type is the request sent from a client application to the GraphQL server. GraphQL uses the Schema Definition Language (SDL) to define a Query.

```
type Query {
   field1: data_type
   field2:data_type

field3(param1:data_type,param2:data_type,...param
N:data_type):data_type
}
```

```
type Query  {
   greeting: String
}
```

Mutation Type

Mutations are operations sent to the server to create, update or delete data. These are analogous to the PUT, POST, PATCH and DELETE verbs to call REST-based APIs.

Mutation is one of the root-level data-types in GraphQL. The Query type defines the entry-points for data-fetching operations whereas the Mutation type specifies the entry points for data-manipulation operations.

```
type Mutation {
  field1: data_type

field2(param1:data_type,param2:data_type,...param
N:data_type):data_type
}
```

```
type Mutation {
  addStudent(firstName: String, lastName: String):
Student
}
```

Enum Type

An Enum is similar to a scalar type. Enums are useful in a situation where the value for a field must be from a prescribed list of options.

The syntax for defining an Enum type is –

```
type enum_name{
   value1
   value2
}
```

Following snippet illustrates how an enum type can be defined –

```
type Days_of_Week{
   SUNDAY
   MONDAY
   TUESDAY
   WEDNESDAY
   THURSDAY
   FRIDAY
   SATURDAY
}
```

List Type

Lists can be used to represent an array of values of specific type. Lists are defined with a type modifier [] that wraps object types, scalars, and enums.

The following syntax can be used to define a list type –

field:[data_type]

The below example defines a list type todos –

```
type Query {
   todos: [String]
}
```

Non-Nullable Type

By default, each of the core scalar types can be set to null. In other words, these types can either return a value of the specified type or they can have no value. To override this default and specify that a field must be defined, an exclamation mark (!) can be appended to a type. This ensures the presence of value in results returned by the query.

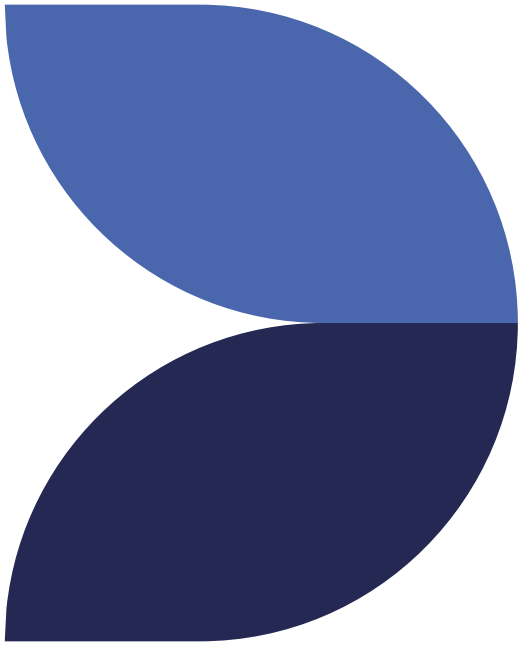
The following syntax can be used to define a non-nullable field –

field:data_type!

stud_id is declared as a mandatory field.

```
type Student {
   stud_id:ID!
   firstName:String
   lastName:String
   fullName:String
   college:College
}
```

Schema



Your GraphQL server uses a **schema** to describe the shape of your available data. This schema defines a hierarchy of **types** with **fields** that are populated from your back-end data stores. The schema also specifies exactly which **queries** and **mutations** are available for clients to execute.

The GraphQL specification defines a human-readable **schema definition language** (or **SDL**) that you use to define your schema and store it as a string.

schema that defines two object types: Book and Author

```
type Book {
 title: String
author: Author
 }

type Author {
name: String
books: [Book]
}
```

A schema defines a collection of types and the relationships *between* those types

Query Type (Revisiting...)

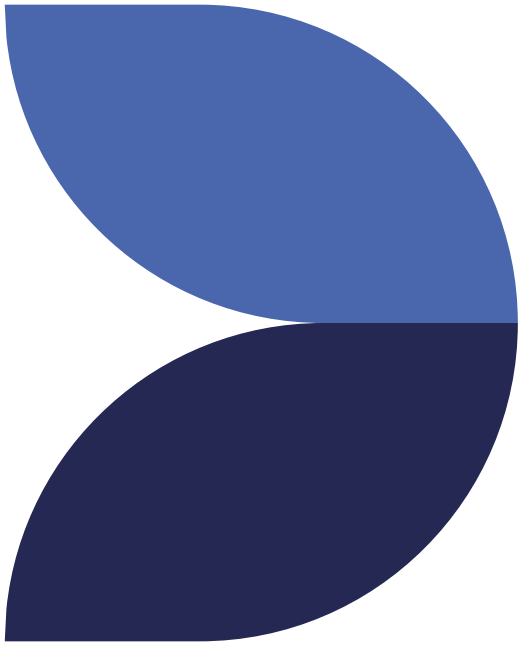
- The Query type is a special object type that defines all of the top-level entry points for queries that clients execute against your server.
- Each field of the Query type defines the name and return type of a different entry point.

```
type Query
{
    books: [Book]
    authors: [Author]
}
```

With a REST-based API, books and authors would probably be returned by different endpoints (e.g., /api/books and /api/authors).

The flexibility of GraphQL enables clients to query both resources with a single request.

Structuring a query



When your clients build queries to execute against your graph, those queries match the shape of the object types you define in your schema.

client could execute the following query, which requests both a list of all book titles *and* a list of all author names:

```
query GetBooksAndAuthors {
    books {
    title
    }

    authors {
    name
    }
}
```

Our server would then respond to the query with results that match the query's structure, like so:

```
{
  "data": {
    "books": [
      {
        "title": "City of Glass"
      },
      ...
    ],
    "authors": [
      {
        "name": "Paul Auster"
      },
      ...
    ]
  }
}
```

Mutation Type (revisiting)

The Mutation type is similar in structure and purpose to the Query type. Whereas the Query type defines entry points for read operations, the Mutation type defines entry points for write operations.

Each field of the Mutation type defines the signature and return type of a different entry point.

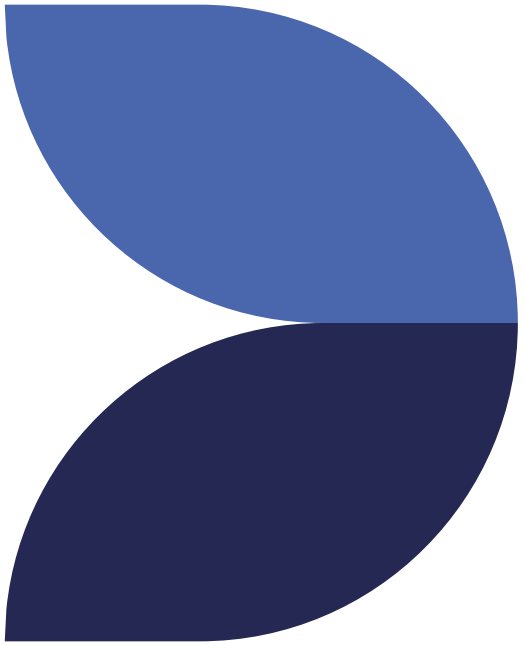
```
type Mutation {

addBook(title: String, author: String): Book

}
```

The mutation accepts two arguments (title and author) and returns a newly created Book object.

Structuring a mutation



Like queries, mutations match the structure of your schema's type definitions. The following mutation creates a new Book and requests certain fields of the created object as a return value

```
mutation CreateBook {
  addBook(title: "Fox in Socks", author: "Dr. Seuss") {
    title
    author {
      name
    }
  }
}
```

As with queries, our server would respond to this mutation with a result that matches the mutation's structure

```
{
  "data": {
    "addBook": {
      "title": "Fox in Socks",
      "author": {
        "name": "Dr. Seuss"
      }
    }
  }
}
```

Enum Type (revisiting)

```
enum AllowedColor {
  RED
  GREEN
  BLUE
}
```

```
type Query {
  favoriteColor: AllowedColor # enum return value
  avatar(borderColor: AllowedColor): String # enum argument
}
```

A query might then look like this:

```
query GetAvatar {
  avatar(borderColor: RED)
}
```

Union type

```
union Media = Book | Movie
```

A field can have a union (or a list of that union) as its return type. In this case, it can return any object type that's included in the union:

```
type Query {
allMedia: [Media] # This list can include both Book and Movie objects
}
```

The following schema defines a SearchResult union type that can return either a Book or an Author:

```
union SearchResult = Book | Author

type Book {
  title: String!
}

type Author {
  name: String!
}

type Query {
  search(contains: String): [SearchResult!]
}
```

The SearchResult union enables Query.search to return a list that includes both Books and Authors.

query for the schema above:

```
query GetSearchResults {
  search(contains: "Shakespeare") {
__typename
    ... on Book {
      title
    }
    ... on Author {
      name
    }
  }
}
```

valid result for the above query:


```
{
  "data": {
    "search": [
      {
        "__typename": "Book",
        "title": "The Complete Works of William Shakespeare"
      },
      {
        "__typename": "Author",
        "name": "William Shakespeare"
      }
    ]
  }
}
```

The __typename field

Every object type in your schema automatically has a field named __typename (you don't need to define it). The __typename field returns the object type's name as a String (e.g., Book or Author).

GraphQL clients use an object's __typename for many purposes, such as to determine which type was returned by a field that can return multiple types (i.e., a union or interface).

Apollo Client relies on __typename when caching results, so it automatically includes __typename in every object of every query.

Because __typename is always present, this is a valid query for any GraphQL server:

```
query UniversalQuery {
  __typename
}
```